

Partie 06 — Écrire ses propres fonctions

INF-STAR — Programmation statistique avec R

Oskar Laverny

October 5, 2026

Aix-Marseille Université

Objectifs

Vous saurez :

- reconnaître une répétition de code qui mérite une fonction ;
- définir des arguments et une valeur de retour ;
- utiliser des valeurs par défaut ;
- écrire une fonction simple et testable ;
- utiliser une fonction anonyme lorsque c'est utile.

Pourquoi écrire une fonction ?



Éviter le copier-coller

Supposons le calcul :

```
poids / taille^2
```

Si on le répète à plusieurs endroits, on risque des incohérences.

On peut donner un nom au calcul.

Première fonction

```
imc <- function(poids, taille) {  
  poids / taille^2  
}
```

Puis :

```
imc(74, 1.72)  
imc(61, 1.65)
```

Anatomie d'une fonction



Arguments, corps, résultat

```
nom <- function(argument1, argument2) {  
  resultat <- argument1 + argument2  
  resultat  
}
```

En R, la dernière expression évaluée est retournée automatiquement.

`return()` est possible

```
imc <- function(poids, taille) {  
  return(poids / taille^2)  
}
```

Pour les fonctions courtes, retourner simplement la dernière expression est souvent plus lisible.

Arguments par défaut

```
centrer <- function(x, na.rm = TRUE) {  
  x - mean(x, na.rm = na.rm)  
}
```

Appels :

```
centrer(c(1, 2, 3))  
centrer(c(1, NA, 3), na.rm = FALSE)
```

Arguments nommés

```
imc(poids = 74, taille = 1.72)
```

Avec plusieurs arguments, les noms rendent le code auto-documenté.

Conditions dans une fonction



Une fonction de catégorisation

```
categorie_imc <- function(x) {  
  if (is.na(x)) {  
    return(NA_character_)  
  }  
  
  if (x < 18.5) {  
    "insuffisance pondérale"  
  } else if (x < 25) {  
    "corpulence normale"  
  } else {  
    "surpoids ou obésité"  
  }  
}
```

Valider des entrées simples

```
imc <- function(poids, taille) {  
  stopifnot(all(poids > 0), all(taille > 0))  
  poids / taille^2  
}
```

Une fonction peut échouer tôt avec un message clair plutôt que produire un résultat absurde.

Portée des variables



Variables locales

```
f <- function(x) {  
  y <- x * 2  
  y + 1  
}
```

`y` est créé pendant l'appel de la fonction. Il n'est pas nécessaire de le créer auparavant dans l'espace global.

Pour ce cours, retenez surtout : une fonction doit dépendre autant que possible de ses **arguments**, pas d'objets cachés dans l'environnement.

```
seuil <- 65

est_senior <- function(age) {
  age >= seuil
}
```

Plus explicite :

```
est_senior <- function(age, seuil = 65) {
  age >= seuil
}
```

Fonctions vectorisées



Une fonction peut accepter un vecteur

```
imc <- function(poids, taille) {  
  poids / taille^2  
}  
  
imc(  
  poids = c(60, 75, 90),  
  taille = c(1.60, 1.75, 1.82)  
)
```

On profite ici des opérations vectorisées de R.

Fonctions anonymes et . . .



Une fonction sans nom

```
function(x) x^2
```

On les utilise lorsqu'une petite fonction ne sert qu'une fois :

```
lapply(list(1:3, 4:6), function(x) mean(x))
```

Nous les retrouverons avec `lapply()` dans la partie suivante. Ce complément peut être repris à ce moment-là, après les exercices sur les fonctions nommées.

Syntaxe courte depuis R 4.1

```
\(x) x^2
```

Exemple :

```
lapply(list(1:3, 4:6), \(x) mean(x))
```

La syntaxe classique `function(x)` reste parfaitement valable et souvent plus explicite pour débiter.

...



Transmettre des arguments supplémentaires

```
resume <- function(x, ...) {  
  mean(x, ...)  
}
```

Puis :

```
resume(c(1, NA, 3), na.rm = TRUE)
```

`...` est utile pour transmettre des options à une autre fonction, mais il n'est pas nécessaire dans toutes les fonctions.

Tester une fonction



Quelques cas représentatifs

Pour `imc()` :

```
imc(70, 1.75)  
imc(c(60, 80), c(1.60, 1.80))
```

Vérifiez :

- un cas simple dont vous connaissez le résultat ;
- un vecteur si la fonction doit être vectorisée ;
- les valeurs manquantes si elles sont possibles ;
- les limites importantes du domaine.

Quand un test révèle un problème

```
centrer_reduire <- function(x) {  
  (x - mean(x, na.rm = TRUE)) / sd(x, na.rm = TRUE)  
}  
  
centrer_reduire(c(4, 4, 4))
```

L'écart-type est nul : le calcul produit des valeurs indéfinies. Avant de corriger le code, décider du comportement attendu : signaler une erreur ou documenter une convention. Un test sert aussi à préciser le contrat.

Atelier — construire progressivement une fonction

1. Effectuer le calcul sur un vecteur concret.
2. Remplacer ce vecteur par un argument.
3. Donner un nom au résultat et le retourner.
4. Tester un deuxième jeu de valeurs.
5. Ajouter une option seulement si elle répond à un besoin.

Exemple : transformer un résumé numérique copié dans plusieurs scripts en `resume_num(x, na.rm = TRUE)`.

À retenir



Une bonne petite fonction

- fait **une chose** claire ;
- a un nom qui décrit cette chose ;
- reçoit ce dont elle a besoin en arguments ;
- renvoie un résultat prévisible ;
- peut être testée indépendamment du reste de l'analyse.

Une fonction n'est pas un exercice abstrait : c'est un outil pour rendre un script plus lisible et plus fiable.